



Universidade Federal de Santa Catarina - UFSC

Introdução a Compiladores - INE5622

Gustavo Cortez de Brito (21202331)

João Vitor Duarte Domingos (21203405)

Manuela Schmitz (20102278)

Víctor Douglas Fernandes (21204918)

Parte 2

FIRST/FOLLOW para a linguagem dada

SUMÁRIO

1. Introdução.....	3
2. Sem tratamento.....	3
3. Fatoração à esquerda.....	4
4. Recursão à esquerda.....	6
5. Problema no ATRIBST.....	7
6. LL(1) final.....	8
7. Tabela de reconhecimento sintático e FIRST FOLLOW.....	10
8. Exemplos de aplicação da tabela de reconhecimento sintático.....	12

1. Introdução

Para facilitar o entendimento de como foi realizada a conversão da gramática para LL1, iremos segregar as produções por técnica de tratamento utilizada.

Em negrito, teremos a gramática original e, logo abaixo, estará a linguagem com as devidas adaptações para leitura da ferramenta de princeton, além da respectiva regra para transformação para LL1.

2. Sem tratamento

Nas produções a seguir foi realizado apenas a conversão da gramática para interpretação pela ferramenta de princeton, sem necessidade de outros tratamentos.

MAIN → **STMT** | **FLIST** | **ε**

MAIN ::= FLIST

MAIN ::= STMT

MAIN ::= "

FDEF → **def id(PARLIST){STMTLIST}**

FDEF ::= def id (PARLIST) { STMTLIST }

FCALL → **id(PARLISTCALL)**

FCALL ::= id (PARLISTCALL)

PRINTST → **print EXPR**

PRINTST ::= print EXPR

STMT → **int VARLIST;** | **ATRIBST;** | **PRINTST;** | **RETURNST;** | **IFSTMT** | **{STMTLIST} ;**

STMT ::= int VARLIST ;

STMT ::= ATRIBST ;

STMT ::= PRINTST ;

STMT ::= RETURNST ;

STMT ::= IFSTMT

STMT ::= { STMTLIST }

STMT ::= ;

FACTOR → **num** | **(NUMEXPR)** | **id**

FACTOR ::= num

FACTOR ::= (NUMEXPR)

FACTOR ::= id

3. Fatoração à esquerda

A fatoração à esquerda consiste em reescrever uma gramática removendo prefixos comuns entre alternativas de uma produção, de forma a eliminar ambiguidades e permitir que o analisador sintático escolha a produção correta com base em apenas um símbolo de lookahead.

FLIST $\rightarrow$ **FDEF FLIST** | **FDEF**

FLIST ::= FDEF FLIST'

FLIST' ::= FDEF FLIST'

FLIST' ::= "

PARLIST $\rightarrow$ **int id, PARLIST** | **int id** | ϵ

PARLIST ::= int id PARLIST'

PARLIST ::= "

PARLIST' ::= , PARLIST

PARLIST' ::= "

VARLIST $\rightarrow$ **id, VARLIST** | **id**

VARLIST ::= id VARLIST'

VARLIST' ::= , VARLIST

VARLIST' ::= "

ATRIBST $\rightarrow$ **id = EXPR** | **id = FCALL**

ATRIBST ::= id := ATRIBST'

ATRIBST' ::= EXPR

ATRIBST' ::= FCALL

PARLISTCALL $\rightarrow$ **id, PARLISTCALL** | **id** | ϵ

PARLISTCALL ::= id PARLISTCALL'

PARLISTCALL ::= "

PARLISTCALL' ::= id PARLISTCALL'

PARLISTCALL' ::= "

RETURNST $\rightarrow$ **return id** | **return**

RETURNST ::= return RETURNST'

RETURNST' ::= id

RETURNST' ::= "

IFSTMT $\rightarrow$ **if(EXPR) {STMT} else {STMT}** | **if(EXPR) {STMT}**

IFSTMT ::= if (EXPR) { STMT } IFSTMT'

IFSTMT' ::= else { STMT }

IFSTMT' ::= "

STMTLIST $\rightarrow$ **STMT** **STMTLIST** | **STMT**
STMTLIST ::= **STMT** **STMTLIST**'
STMTLIST' ::= **STMT** **STMTLIST**'
STMTLIST' ::= "

EXPR $\rightarrow$ **NUMEXPR** **<** **NUMEXPR**
| **NUMEXPR** **<=** **NUMEXPR**
| **NUMEXPR** **>** **NUMEXPR**
| **NUMEXPR** **>=** **NUMEXPR**
| **NUMEXPR** **==** **NUMEXPR**
| **NUMEXPR** **!=** **NUMEXPR**
| **NUMEXPR**
EXPR ::= **NUMEXPR** **EXPR'**
EXPR' ::= **<** **NUMEXPR**
EXPR' ::= **<=** **NUMEXPR**
EXPR' ::= **>** **NUMEXPR**
EXPR' ::= **>=** **NUMEXPR**
EXPR' ::= **==** **NUMEXPR**
EXPR' ::= **!=** **NUMEXPR**
EXPR' ::= "

4. Recursão à esquerda

Eliminamos a recursão à esquerda reescrevendo as produções da gramática, de forma que o símbolo não terminal não apareça no início da parte direita da mesma regra, evitando assim derivação recursiva imediata.

NUMEXPR $\rightarrow$ **NUMEXPR + TERM | NUMEXPR - TERM | TERM**

NUMEXPR ::= **TERM NUMEXPR'**

NUMEXPR' ::= **+ TERM NUMEXPR'**

NUMEXPR' ::= **- TERM NUMEXPR'**

NUMEXPR' ::= **"**

TERM $\rightarrow$ **TERM * FACTOR**

TERM ::= **FACTOR TERM'**

TERM' ::= *** FACTOR TERM'**

TERM' ::= **/ FACTOR TERM'**

TERM' ::= **"**

5. Problema no ATRIBST

Para resolver o conflito LL(1) na regra ATRIBST por meio da fatoração à esquerda, precisamos analisar as produções de ATRIBST' e identificar os prefixos comuns que causam ambiguidade.

O problema em ATRIBST' surge porque tanto EXPR (através de FACTOR ::= id) quanto FCALL podem começar com o terminal id.

Se ATRIBST' escolhe EXPR, e EXPR se resolve em id, o lookahead é id.

Se ATRIBST' escolhe FCALL, que é id (PARLISTCALL) (ex: a := func();), o lookahead também é id. Após refatoração:

```
ATRIBST ::= id := ATRIBST'  
ATRIBST' ::= num TERM' NUMEXPR' EXPR'  
ATRIBST' ::= ( NUMEXPR ) TERM' NUMEXPR' EXPR'  
ATRIBST' ::= id ATRIBST_ID_SUFFIX  
ATRIBST_ID_SUFFIX ::= TERM' NUMEXPR' EXPR'  
ATRIBST_ID_SUFFIX ::= ( PARLISTCALL )
```

A regra ATRIBST ::= id := ATRIBST' permanece a mesma. As produções de ATRIBST' agora são distintas com base no primeiro token:

- Se o token após := for num, a primeira produção de ATRIBST' é escolhida. (Corresponde a ATRIBST' ::= EXPR onde EXPR inicia com num).
- Se o token for (, a segunda produção é escolhida. (Corresponde a ATRIBST' ::= EXPR onde EXPR inicia com ().
- Se o token for id, a terceira produção (id ATRIBST_ID_SUFFIX) é escolhida.

O novo não-terminal ATRIBST_ID_SUFFIX decide o que fazer após o id ter sido consumido (no lado direito de uma atribuição):

- Se o token seguinte for (, então o id era o início de uma chamada de função (FCALL). A produção ATRIBST_ID_SUFFIX ::= (PARLISTCALL) é escolhida.
- Caso contrário (se o token seguinte for um operador como *, +, <, etc., ou um terminador como ;), o id era uma variável usada numa expressão (ou apenas a própria variável). A produção ATRIBST_ID_SUFFIX ::= TERM' NUMEXPR' EXPR' é escolhida. Se essa sequência (TERM' NUMEXPR' EXPR') derivar em vazio (epsilon), significa que a expressão era apenas o id.

6. LL(1) final

Esta foi a linguagem resultado final em LL(1) utilizada na ferramenta de princeton (<https://www.cs.princeton.edu/courses/archive/spring20/cos320/LL1/>) para geração da tabela FIRST FOLLOW:

```
MAIN ::= FLIST
MAIN ::= STMT
MAIN ::= "
FLIST ::= FDEF FLIST'
FLIST' ::= FDEF FLIST'
FLIST' ::= "
FDEF ::= def id ( PARLIST ) { STMTLIST }
PARLIST ::= int id PARLIST'
PARLIST ::= "
PARLIST' ::= , PARLIST
PARLIST' ::= "
VARLIST ::= id VARLIST'
VARLIST' ::= , VARLIST
VARLIST' ::= "
STMT ::= int VARLIST ;
STMT ::= ATRIBST ;
STMT ::= PRINTST ;
STMT ::= RETURNST ;
STMT ::= IFSTMT
STMT ::= { STMTLIST }
STMT ::= ;
ATRIBST ::= id := ATRIBST'
ATRIBST' ::= num TERM' NUMEXPR' EXPR'
ATRIBST' ::= ( NUMEXPR ) TERM' NUMEXPR' EXPR'
ATRIBST' ::= id ATRIBST_ID_SUFFIX
ATRIBST_ID_SUFFIX ::= TERM' NUMEXPR' EXPR'
ATRIBST_ID_SUFFIX ::= ( PARLISTCALL )
PARLISTCALL ::= id PARLISTCALL'
PARLISTCALL ::= "
PARLISTCALL' ::= id PARLISTCALL'
PARLISTCALL' ::= "
PRINTST ::= print EXPR
RETURNST ::= return RETURNST'
RETURNST' ::= id
RETURNST' ::= "
IFSTMT ::= if ( EXPR ) { STMT } IFSTMT'
IFSTMT' ::= else { STMT }
IFSTMT' ::= "
```


STMTLIST ::= STMT STMTLIST'
STMTLIST' ::= STMT STMTLIST'
STMTLIST' ::= "
EXPR ::= NUMEXPR EXPR'
EXPR' ::= < NUMEXPR
EXPR' ::= <= NUMEXPR
EXPR' ::= > NUMEXPR
EXPR' ::= >= NUMEXPR
EXPR' ::= == NUMEXPR
EXPR' ::= != NUMEXPR
EXPR' ::= "
NUMEXPR ::= TERM NUMEXPR'
NUMEXPR' ::= + TERM NUMEXPR'
NUMEXPR' ::= - TERM NUMEXPR'
NUMEXPR' ::= "
TERM ::= FACTOR TERM'
TERM' ::= * FACTOR TERM'
TERM' ::= / FACTOR TERM'
TERM' ::= "
FACTOR ::= num
FACTOR ::= (NUMEXPR)
FACTOR ::= id

Tabela para visualização na íntegra:
https://drive.google.com/file/d/1yjcW0dQF_zA4fjstbQ9qEzLwQGkBXACS/view?usp=sharing

8. Exemplos de aplicação da tabela de reconhecimento sintático

Nos três exemplos abaixo podemos notar que a fila está vazia, portanto, o exemplo foi aceito.

Token stream separated by spaces:

def id () { int id ; }

Start/Reset

Step Forward

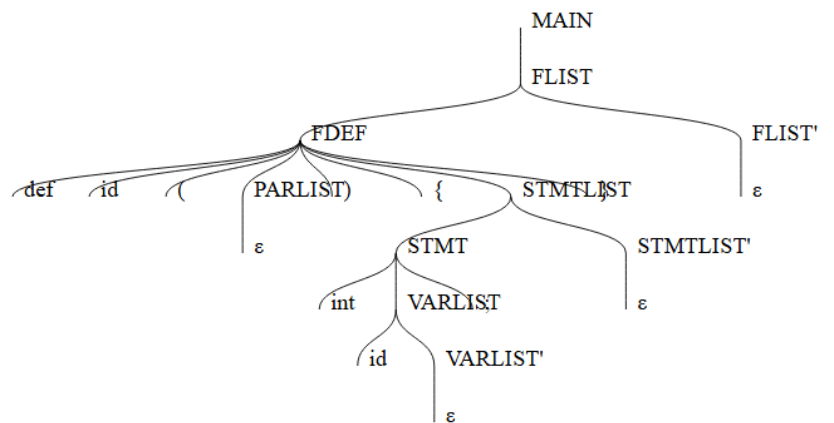
Stack

Remaining Input

Rule

Match \$

Partial Parse Tree



Token stream separated by spaces:

return id ;

Start/Reset

Step Forward

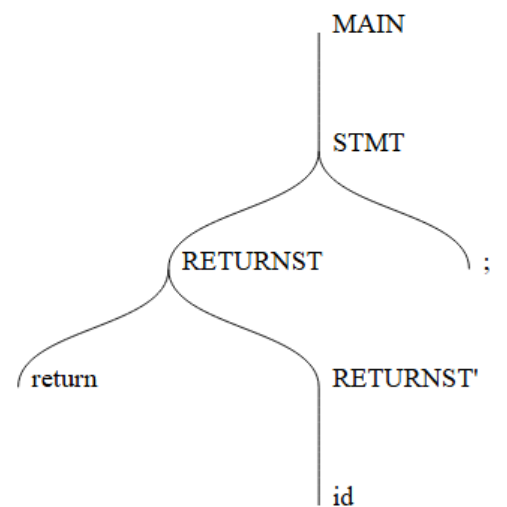
Stack

Remaining Input

Rule

Match \$

Partial Parse Tree



Token stream separated by spaces: id := id != id ;

[Start/Reset](#) [Step Forward](#)

Stack

Remaining Input

Rule

Match \$

Partial Parse Tree

